

Міністерство освіти і науки України
Національний університет "Львівська політехніка"



Звіт до лабораторної роботи №6
З дисципліни "Web-технології та Web-дизайн"
на тему: "React.js: Home page"

Виконала:

Ст.гр. ІР-11

Луцан І.З.

Прийняв:

доцент, к.т.н.

Іванюк О.О.

Львів 2025

Завдання:

6. React.js: Home page

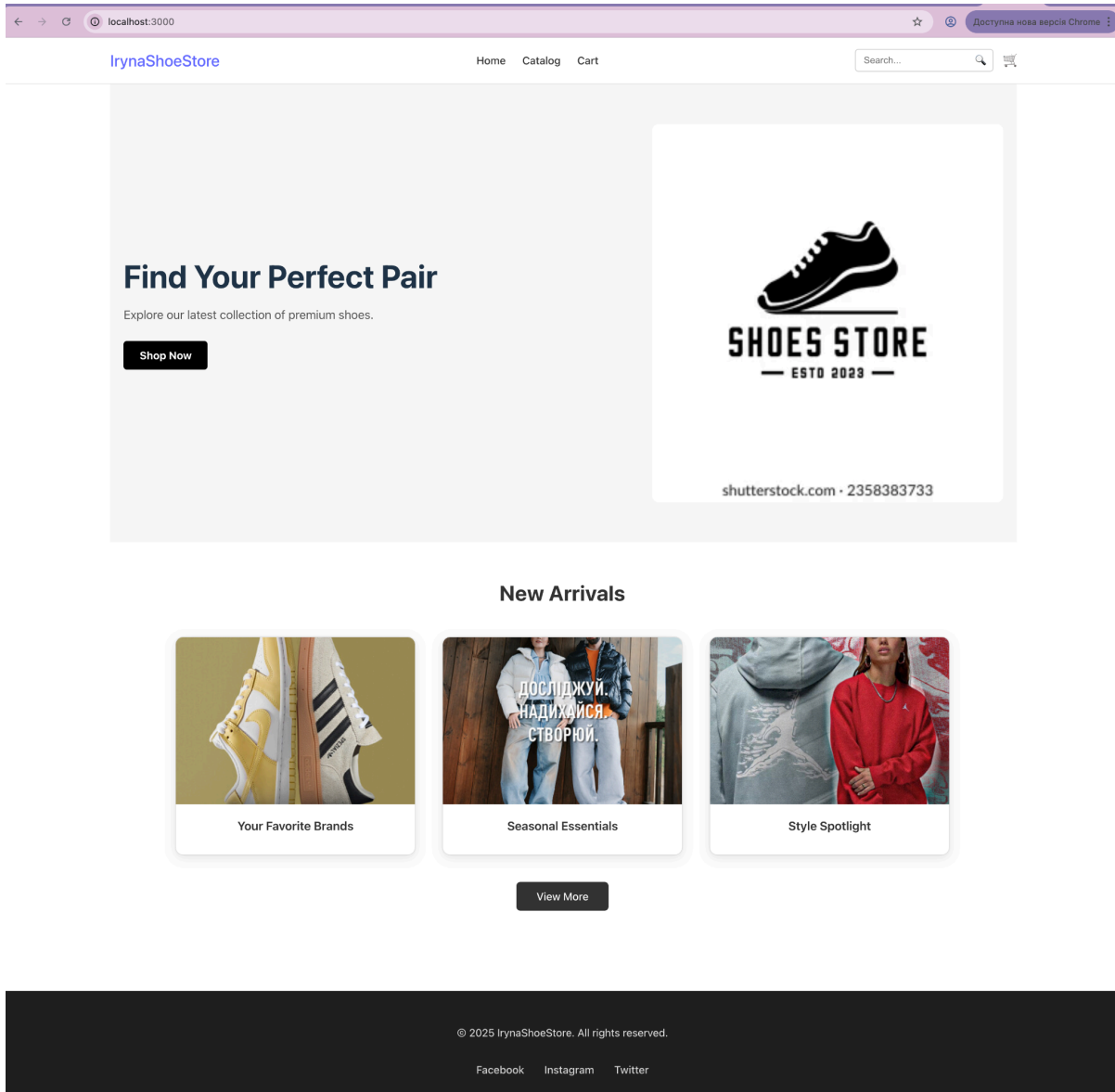
Description: Start creating your React App with a simple Home page (see the link to wireframe above). Your e-commerce app subject is about your **entities from previous (3-5) works.**

Variants - *(products that you are 'selling') the same as for previous works. (see the description to 3rd work)*

Requirements:

- You have to use only React.js library for all of your mark-up. Which means - your index.html file shouldn't be touched.
- Use **create-react-app** tool for creating base of your project
<https://github.com/facebook/create-react-app>
- Your Home page should **follow the wireframe**, i.e all of the elements (header, navigation, footer etc.) from wireframes **must be** presented on your website.
- **Design:** You have to use **CSS** styling or/and component libraries. Don't overthink it, use your imagination for UI, BUT the work with almost no styling is unacceptable.
- **Functionality:** For this work nothing but view only part is required. Any interaction (links/buttons) is not necessary, but you will have to complete them in your next labs.
- **Code style:**
 - Project structure: Your UI elements should be logically separated into React components (one file for each component) - at least **5 component files are required**. In other words, you can't just put all your JSX into 1-2 files.
 - Use **Functional components** instead of **Class components**
<https://uk.reactjs.org/docs/components-and-props.html>

Сайт:



Реалізація бекенду:

файл [app.py](#):

```
import os
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy
from flask_cors import CORS

# Ініціалізація Flask
app = Flask(__name__, template_folder='templates', static_folder='static')
CORS(app)

basedir = os.path.abspath(os.path.dirname(__file__))
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:/// ' + os.path.join(basedir,
'shoes.db')
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

```

db = SQLAlchemy(app)

# --- Модель даних для взуття (Таблиця) ---
class Shoe(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    producer = db.Column(db.String(100), nullable=False)
    price = db.Column(db.Float, nullable=False)
    size = db.Column(db.Integer, nullable=False)
    color = db.Column(db.String(50), nullable=False)

    def to_dict(self):
        return {
            'id': self.id,
            'producer': self.producer,
            'price': self.price,
            'size': self.size,
            'color': self.color
        }

# --- Модель даних для новин (Таблиця) ---
class FeaturedNews(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    image = db.Column(db.String(255), nullable=False) # Шлях до зображення
    title = db.Column(db.String(100), nullable=False)

    def to_dict(self):
        return {
            'id': self.id,
            'image': self.image,
            'title': self.title
        }

# CRUD Операції для взуття (API Endpoints)

# CREATE (Створити взуття)
@app.route('/api/shoes', methods=['POST'])
def add_shoe():
    data = request.get_json()

    if not all(k in data for k in ('producer', 'price', 'size', 'color')):
        return jsonify({'error': 'Missing data'}), 400

    try:
        new_shoe = Shoe(
            producer=data['producer'],
            price=float(data['price']),
            size=int(data['size']),
            color=data['color']
        )
        db.session.add(new_shoe)

```

```

        db.session.commit()
        return jsonify(new_shoe.to_dict()), 201
    except (ValueError, TypeError):
        return jsonify({'error': 'Invalid data format'}), 400

# READ (Отримати всі взуття)
@app.route('/api/shoes', methods=['GET'])
def get_shoes():
    shoes = Shoe.query.all()
    return jsonify([shoe.to_dict() for shoe in shoes])

# READ (Отримати одне взуття)
@app.route('/api/shoes/<int:id>', methods=['GET'])
def get_shoe(id):
    shoe = db.session.get(Shoe, id)
    if shoe:
        return jsonify(shoe.to_dict())
    else:
        return jsonify({'error': 'Shoe not found'}), 404

# UPDATE (Оновити взуття)
@app.route('/api/shoes/<int:id>', methods=['PUT'])
def update_shoe(id):
    shoe = db.session.get(Shoe, id)
    if not shoe:
        return jsonify({'error': 'Shoe not found'}), 404

    data = request.get_json()

    try:
        shoe.producer = data.get('producer', shoe.producer)
        shoe.price = float(data.get('price', shoe.price))
        shoe.size = int(data.get('size', shoe.size))
        shoe.color = data.get('color', shoe.color)

        db.session.commit()
        return jsonify(shoe.to_dict())
    except (ValueError, TypeError):
        return jsonify({'error': 'Invalid data format'}), 400

# DELETE (Видалити взуття)
@app.route('/api/shoes/<int:id>', methods=['DELETE'])
def delete_shoe(id):
    shoe = db.session.get(Shoe, id)
    if shoe:
        db.session.delete(shoe)
        db.session.commit()
        return jsonify({'message': 'Shoe deleted successfully'})
    else:
        return jsonify({'error': 'Shoe not found'}), 404

```

```
# --- API для новин ---

# CREATE (Додати новину)
@app.route('/api/featured-news', methods=['POST'])
def add_featured_news():
    data = request.get_json()

    if not all(k in data for k in ('image', 'title')):
        return jsonify({'error': 'Missing data'}), 400

    try:
        new_news = FeaturedNews(
            image=data['image'],
            title=data['title']
        )
        db.session.add(new_news)
        db.session.commit()
        return jsonify(new_news.to_dict()), 201
    except Exception as e:
        return jsonify({'error': str(e)}), 400

# READ (Отримати всі новини)
@app.route('/api/featured-news', methods=['GET'])
def get_featured_news():
    news = FeaturedNews.query.all()
    return jsonify([news_item.to_dict() for news_item in news])

# READ (Отримати одну новину)
@app.route('/api/featured-news/<int:id>', methods=['GET'])
def get_featured_news_item(id):
    news_item = db.session.get(FeaturedNews, id)
    if news_item:
        return jsonify(news_item.to_dict())
    else:
        return jsonify({'error': 'News not found'}), 404

# UPDATE (Оновити новину)
@app.route('/api/featured-news/<int:id>', methods=['PUT'])
def update_featured_news(id):
    news_item = db.session.get(FeaturedNews, id)
    if not news_item:
        return jsonify({'error': 'News not found'}), 404

    data = request.get_json()

    try:
        news_item.image = data.get('image', news_item.image)
        news_item.title = data.get('title', news_item.title)
```

```

        db.session.commit()
        return jsonify(news_item.to_dict())
    except Exception as e:
        return jsonify({'error': str(e)}), 400

# DELETE (Видалити новину)
@app.route('/api/featured-news/<int:id>', methods=['DELETE'])
def delete_featured_news(id):
    news_item = db.session.get(FeaturedNews, id)
    if news_item:
        db.session.delete(news_item)
        db.session.commit()
        return jsonify({'message': 'News deleted successfully'})
    else:
        return jsonify({'error': 'News not found'}), 404

# Функція для додавання початкових даних
def add_mock_data():
    # Перевіряємо, чи вже є дані в таблиці
    if FeaturedNews.query.count() == 0:
        initial_news = [
            FeaturedNews(
                image="src/assets/img1.jpeg",
                title="Your Favorite Brands"
            ),
            FeaturedNews(
                image="src/assets/img2.jpeg",
                title="Seasonal Essentials"
            ),
            FeaturedNews(
                image="src/assets/img3.jpeg",
                title="Style Spotlight"
            )
        ]

        for news_item in initial_news:
            db.session.add(news_item)

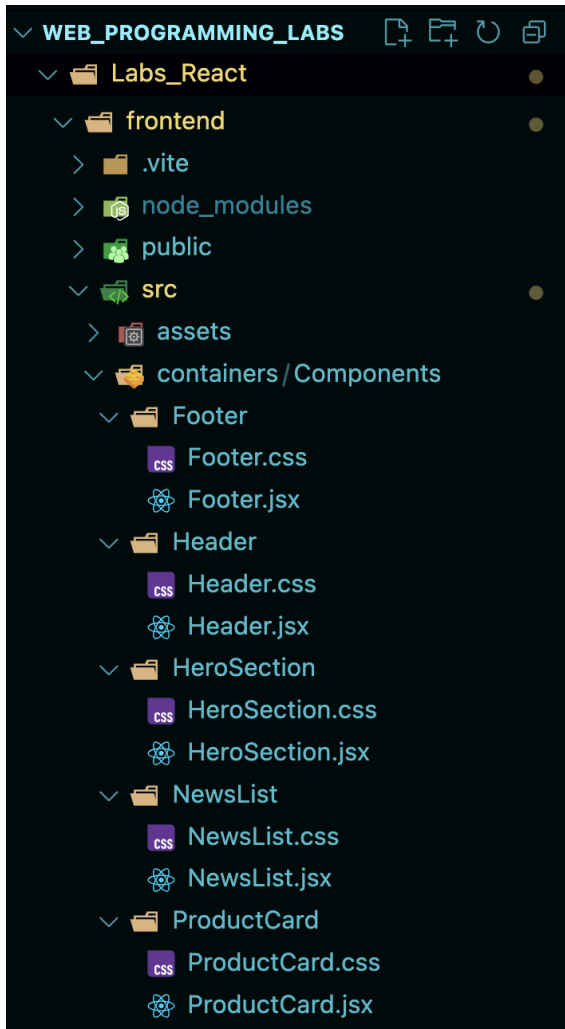
        db.session.commit()
        print("Initial featured news added to database.")

# --- Запуск сервера ---
if __name__ == '__main__':
    # Створюємо таблиці, якщо їх ще немає
    with app.app_context():
        db.create_all()
        add_mock_data() # Додаємо початкові дані

    # Запускаємо Flask-сервер
    app.run(port=5001, debug=True)

```

Структура фронтенду:



Файл App.jsx:

```
import './App.css';
import Header from './containers/Components/Header/Header.jsx';
import HeroSection from './containers/Components/HeroSection/HeroSection.jsx';
import NewsList from './containers/Components/NewsList/NewsList.jsx';
import Footer from './containers/Components/Footer/Footer.jsx';

function App() {
  return (
    <div className="App">
      <Header />
      <HeroSection />
      <NewsList />
      <Footer />
    </div>
  );
}

export default App;
```

Файл main.jsx:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Посилання на лабораторну роботу на гітхабі:

https://github.com/IrynaLutsan/Web_Programming_Labs/tree/main/Labs_React

Висновок:

У межах лабораторної роботи було створено React-застосунок із дотриманням вимоги працювати виключно з React-компонентами. Головну сторінку реалізовано відповідно до наданого вайрфрейму, додано необхідні елементи інтерфейсу та виконано базове стилювання за допомогою CSS. Увесь інтерфейс був логічно поділений на окремі функціональні компоненти, що покращило структуру проекту та читабельність коду.